1 2 9 0 FACULDADE DE CIÊNCIAS E TECNOLOGIA UNIVERSIDADE D COIMBRA

deer.uc DEPARTAMENTO DE ENGENHARIA ELETROTÉCNICA E DE COMPUTADORES

Master's Degree in Electrical and Computer Engineering

DIGITAL SYSTEM PROJECT

PROJECT TITLE

GUITAR HERO Recreation on FPGA using VHDL

Mark John Ensa – 2024251008
Hina Iqbal - 2023182290

Introduction

The purpose of this project is to recreate the guitar hero video game in an FPGA using VHDL. This will be done by using VGA to display the game and using the four buttons on the DE2-115 board to control the game. The actual game will consist of 4 rows of notes approaching the bottom middle of the screen, corresponding to the four buttons on the DE2-115 board. When the user starts the game by pressing any button, different notes will scroll down the screen and the user must press the corresponding buttons at the same time as when the note crosses a certain threshold. The song that the notes correspond to is the ABCs. If the player hits the note correctly, the game will increase the score of the player by one. This score will be displayed on the 7-segment display while the game is going along, showing your final score at the end. It also aims to recreate the popular video game *Guitar Hero* on an FPGA platform using VHDL. The implementation utilizes the DE2-115 development board, leveraging its VGA output for game display and four onboard buttons for user interaction. The project demonstrates the integration of digital design, hardware description languages, and real-time user input handling to deliver an interactive gaming experience reminiscent of the original Guitar Hero

Project Objectives

- Recreate the core gameplay mechanics of Guitar Hero on FPGA hardware.
- Display game graphics using VGA.
- Map four game note lanes to four physical buttons on the DE2-115 board.
- Implement real-time score tracking and display using a seven-segment display.
- Provide a replayable, responsive game loop.

System Overview

Gameplay Mechanics:

- The game starts with an introduction screen and a score display initialized to 00.
- Flipping switch SW0 starts the game, triggering notes to scroll down the screen in sync with the melody of the ABCs song.
- Players must press the corresponding button as a note crosses a threshold at the bottom of the screen.
- Correct timing increases the score by one; incorrect presses subtract one point.
- The maximum achievable score is 26, matching the number of notes in the ABCs song.
- The game can be restarted indefinitely by toggling the introduction screen switch

User Interface:

- VGA Output: Displays the game screen, notes, and movement.
- Seven-Segment Display: Shows the current score in real time.
- Control Buttons: Four buttons correspond to four note lanes.
- Menu Switch: Used to start/restart the game.

Hardware and VHDL Design

VGA Controller

A custom VHDL module (vga_controller.vhd) generates the necessary VGA synchronization signals and manages pixel coordinates for display rendering. Key parameters include horizontal and vertical sync pulse widths, porch timings, and display resolutions. The controller ensures correct timing for VGA output, enabling smooth graphics rendering.

Image Generator

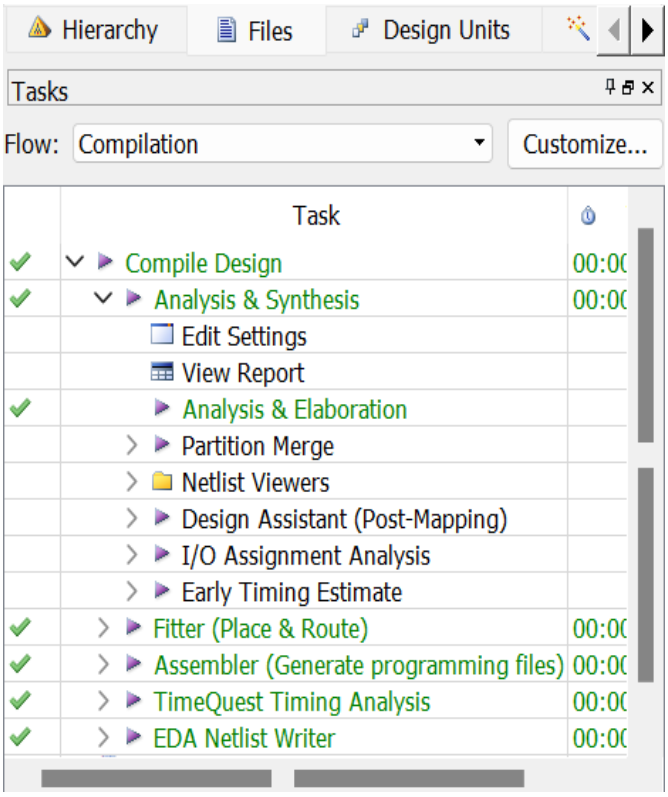
The hw_image_generator.vhd module is responsible for:

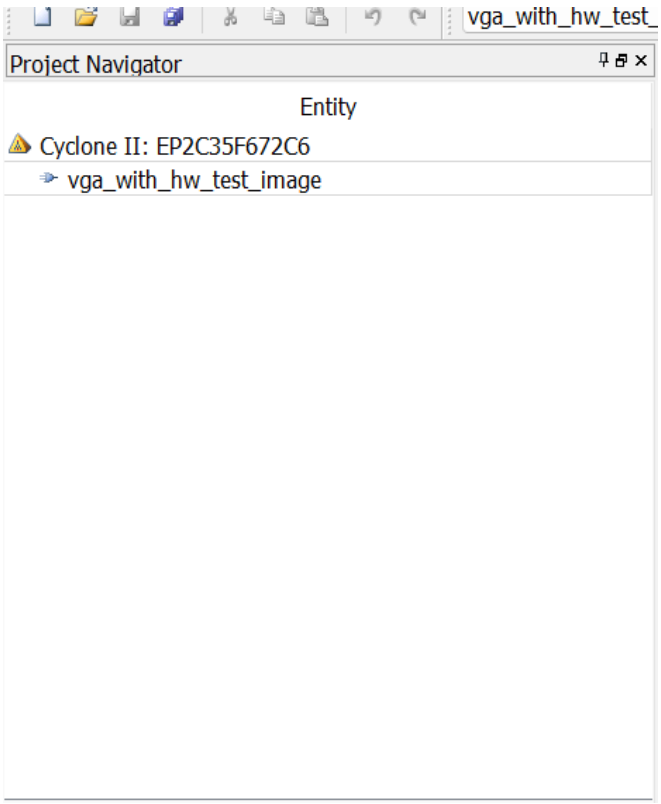
- Rendering the game background, note lanes, and notes in their respective colors (green, red, yellow, blue).
- Managing the movement of notes by updating their positions each clock cycle.
- Detecting button presses and updating the score accordingly.
- Outputting RGB values to the VGA DAC and score values to the seven-segment display.

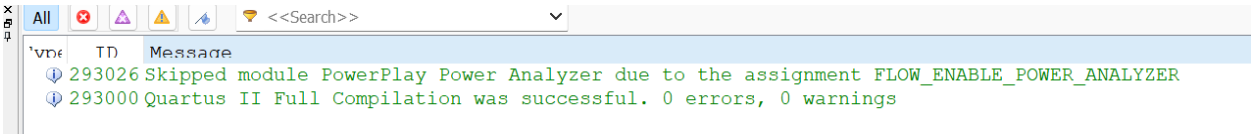
Project Operation

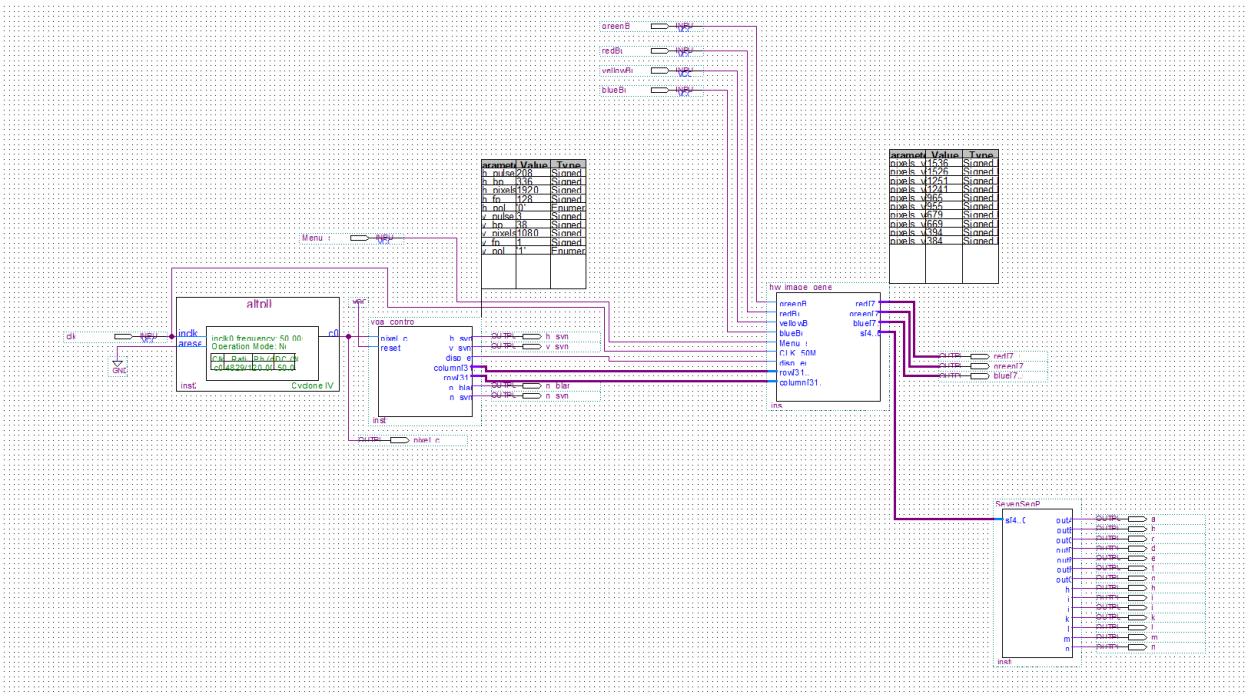
The game opens to the Guitar Hero introduction screen and the score displays 00. To start the game, the first switch SW0 needs to be flipped. Next, the notes will start to approach the threshold gray bars at the bottom of the screen to the tune and pattern of the ABCs. While the notes are within the threshold, the respective button needs to be pressed to gain points and add to the score. If a note is pressed incorrectly, one point is subtracted from the score. A perfect score would be 26. If no buttons are pressed, the score will not change. When the game concludes, it can be restarted by flipping the introduction screen switch and starting the game and scoring over so that the user can play an infinite amount of time.

- Features completely working
    - Title screen
    - Game screen, notes and their movement
    - Score Display (seven segment display)
- Features partially working
    - Menu switch
        - When you flip the switch very fast multiple times often notes will disappear from the game screen
    - Buttons (specifically debouncing)
        - Might still be possible to hit the button multiple times within a note


Hierarchy
Files
Design Units
Tasks
Flow: Compilation
Customize...
Task
Compile Design
Analysis & Synthesis
Edit Settings
View Report
Analysis & Elaboration
Partition Merge
Netlist Viewers
Design Assistant (Post-Mapping)
I/O Assignment Analysis
Early Timing Estimate
Fitter (Place & Route)
Assembler (Generate programming files)
TimeQuest Timing Analysis
EDA Netlist Writer


vga_with_hw_test_
Project Navigator
Entity
Cyclone II: EP2C35F672C6
vga_with_hw_test_image


All
<<Search>>
Message
293026 Skipped module PowerPlay Power Analyzer due to the assignment FLOW_ENABLE_POWER_ANALYZER
293000 Quartus II Full Compilation was successful. 0 errors, 0 warnings



Future Suggestions

Future versions of the project should implement some kind of music or sound to accompany the game. The auditory feature of the game is an important one that could not be included during the span of this project but would increase the complexity and accuracy of the game as compared to the real Guitar Hero video game. Other more feasible additions could be additional levels with different songs connected to different switches or changes in the difficulty of song choices. Also, using the LCD to display song titles or other messages would add a new level of detail for the user.

Conclusion

This project successfully demonstrates the feasibility of implementing a simplified Guitar Hero game on FPGA hardware using VHDL. Core gameplay, graphics, and real-time score tracking is operational, providing a functional prototype. Addressing the current limitations and expanding the feature set would further enhance the educational and entertainment value of the project

Appendix

VGA_Controller.vhd

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY vga_controller IS
        GENERIC(
                h_pulse    :    INTEGER := 208;          --horiztonal sync pulse width in pixels
                h_bp       :    INTEGER := 336;          --horizontal back porch width in pixels
                h_pixels   :    INTEGER := 1920;         --horizontal display width in pixels
                h_fp       :    INTEGER := 128;          --horizontal front porch width in pixels
                h_pol      :    STD_LOGIC := '0';        --horizontal sync pulse polarity (1 = positive, 0 = negative)
                v_pulse    :    INTEGER := 3;            --vertical sync pulse width in rows
                v_bp       :    INTEGER := 38;           --vertical back porch width in rows
                v_pixels   :    INTEGER := 1080;         --vertical display width in rows
                v_fp       :    INTEGER := 1;            --vertical front porch width in rows
                v_pol      :    STD_LOGIC := '1');       --vertical sync pulse polarity (1 = positive, 0 = negative)
        PORT(
                pixel_clk  :    IN     STD_LOGIC;        --pixel clock at frequency of VGA mode being used
                reset_n    :    IN     STD_LOGIC;        --active low asycnchronous reset
                h_sync     :    OUT    STD_LOGIC;        --horiztonal sync pulse
                v_sync     :    OUT    STD_LOGIC;        --vertical sync pulse
                disp_ena   :    OUT    STD_LOGIC;        --display enable ('1' = display time, '0' = blanking time)
                column     :    OUT    INTEGER;          --horizontal pixel coordinate
                row        :    OUT    INTEGER;          --vertical pixel coordinate
```

```
                n_blank    :        OUT     STD_LOGIC;          --direct blacking output to DAC
                n_sync     :        OUT     STD_LOGIC);         --sync-on-green output to DAC
END vga_controller;

ARCHITECTURE behavior OF vga_controller IS
        CONSTANT        h_period   :        INTEGER := h_pulse + h_bp + h_pixels + h_fp;  --total number of pixel clocks in a row
        CONSTANT        v_period   :        INTEGER := v_pulse + v_bp + v_pixels + v_fp;  --total number of rows in column
BEGIN

        n_blank <= '1';  --no direct blanking
        n_sync <= '0';   --no sync on green

        PROCESS(pixel_clk, reset_n)
                VARIABLE h_count  :        INTEGER RANGE 0 TO h_period - 1 := 0;  --horizontal counter (counts the columns)
                VARIABLE v_count  :        INTEGER RANGE 0 TO v_period - 1 := 0;  --vertical counter (counts the rows)
        BEGIN

                IF(reset_n = '0') THEN                  --reset asserted
                        h_count := 0;                   --reset horizontal counter
                        v_count := 0;                   --reset vertical counter
                        h_sync <= NOT h_pol;            --deassert horizontal sync
                        v_sync <= NOT v_pol;            --deassert vertical sync
                        disp_ena <= '0';                --disable display
                        column <= 0;                    --reset column pixel coordinate
                        row <= 0;                       --reset row pixel coordinate

                ELSIF(pixel_clk'EVENT AND pixel_clk = '1') THEN

                        --counters
                        IF(h_count < h_period - 1) THEN                 --horizontal counter (pixels)
                                h_count := h_count + 1;
                        ELSE
                                h_count := 0;
                                IF(v_count < v_period - 1) THEN         --veritcal counter (rows)
                                        v_count := v_count + 1;
                                ELSE
                                        v_count := 0;
                                END IF;
                        END IF;

                        --horizontal sync signal
                        IF(h_count < h_pixels + h_fp OR h_count >= h_pixels + h_fp + h_pulse) THEN
                                h_sync <= NOT h_pol;            --deassert horiztonal sync pulse
                        ELSE
                                h_sync <= h_pol;                --assert horiztonal sync pulse
                        END IF;

                        --vertical sync signal
                        IF(v_count < v_pixels + v_fp OR v_count >= v_pixels + v_fp + v_pulse) THEN
                                v_sync <= NOT v_pol;            --deassert vertical sync pulse
                        ELSE
                                v_sync <= v_pol;                --assert vertical sync pulse
                        END IF;

                        --set pixel coordinates
                        IF(h_count < h_pixels) THEN     --horiztonal display time
                                column <= h_count;      --set horiztonal pixel coordinate
                        END IF;
                        IF(v_count < v_pixels) THEN     --vertical display time
                                row <= v_count;         --set vertical pixel coordinate
                        END IF;
```

```
                --set display enable output
                IF(h_count < h_pixels AND v_count < v_pixels) THEN          --display time
                        disp_ena <= '1';
                 --enable display
                ELSE
                --blanking time
                        disp_ena <= '0';
                        --disable display
                END IF;

        END IF;
    END PROCESS;

END behavior;
```

Hw_image_generator.vhd

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;
use IEEE.STD_LOGIC_ARITH.all;
use IEEE.STD_LOGIC_UNSIGNED.all;
use ieee.numeric_std.all;

ENTITY hw_image_generator IS
    GENERIC(
        pixels_y10: INTEGER := 1536;
        pixels_y9: INTEGER := 1526;
        pixels_y8: INTEGER := 1251;
        pixels_y7: INTEGER := 1241;
        pixels_y6: INTEGER := 965;
        pixels_y5: INTEGER := 955;
        pixels_y4: INTEGER := 679;
        pixels_y3: INTEGER := 669;
        pixels_y2: INTEGER := 394;
        pixels_y :  INTEGER := 384 );
    PORT(
        greenBut        :       IN      STD_LOGIC;
        redBut          :       IN      STD_LOGIC;
        yellowBut       :       IN      STD_LOGIC;
        blueBut         :       IN      STD_LOGIC;
        Menu_sw         :       IN      STD_LOGIC;
        CLK_50MHz       :       IN      STD_LOGIC;
        disp_ena        :       IN      STD_LOGIC;      --display enable ('1' = display time, '0' = blanking time)
        row             :       IN      INTEGER;        --row pixel coordinate
        column          :       IN      INTEGER;        --column pixel coordinate
        red     :       OUT     STD_LOGIC_VECTOR(7 DOWNTO 0) := (OTHERS => '0');  --red magnitude output to DAC
        green   :       OUT     STD_LOGIC_VECTOR(7 DOWNTO 0) := (OTHERS => '0');  --green magnitude output to DAC
        blue    :       OUT     STD_LOGIC_VECTOR(7 DOWNTO 0) := (OTHERS => '0');  --blue magnitude output to DAC
        s       :       OUT     STD_LOGIC_VECTOR(4 DOWNTO 0) := (OTHERS => '0')); --seven segment display output

END hw_image_generator;

ARCHITECTURE behavior OF hw_image_generator IS

    signal counter : std_logic_vector(24 downto 0); --signal for counting
    ------------------------------------------------------------- Green Note Thresholds
    signal green_1      :  INTEGER  := -1510;
    signal green_2      :  INTEGER  := -1620;
    signal green_3      :  INTEGER  := -1730;
    signal green_4      :  INTEGER  := -1950;
    signal green_5      :  INTEGER  := -2570;
```

```
signal green_6          : INTEGER  := -3190;
signal green_7          : INTEGER  := -3990;
signal green_8          : INTEGER  := -4190;
------------------------------------------------------------ Red Note Thresholds
signal red_1            : INTEGER  := 0;
signal red_2            : INTEGER  := -110;
signal red_3            : INTEGER  := -1190;
signal red_4            : INTEGER  := -1300;
signal red_5            : INTEGER  := -1840;
signal red_6            : INTEGER  := -2360;
signal red_7            : INTEGER  := -2980;
signal red_8            : INTEGER  := -3690;
------------------------------------------------------------ Yellow Note Thresholds
signal yellow_1         : INTEGER  := -230;
signal yellow_2         : INTEGER  := -340;
signal yellow_3         : INTEGER  := -670;
signal yellow_4         : INTEGER  := -970;
signal yellow_5         : INTEGER  := -1080;
signal yellow_6         : INTEGER  := -2250;
signal yellow_7         : INTEGER  := -2870;
signal yellow_8         : INTEGER  := -3590;
------------------------------------------------------------ Blue Note thresholds
signal blue_1           : INTEGER  := -450;
signal blue_2           : INTEGER  := -560;
------------------------------------------------------------
signal noteWidth        :          INTEGER := 50; -- Width of all notes
signal currScore        :          std_logic_vector(3 downto 0) := (OTHERS => '0'); -- Score of the player
signal add1                        :          INTEGER := 1; -- Value that output "s" increments by
signal greenBut_debounced          :          STD_LOGIC          := '0'; -- Green button debouncing signal
signal redBut_debounced            :          STD_LOGIC          := '0'; -- Red button debouncing signal
signal yellowBut_debounced         :          STD_LOGIC          := '0'; -- Yellow button debouncing signal
signal blueBut_debounced           :          STD_LOGIC          := '0'; -- Blue button debouncing signal
signal key_pressed                 :          STD_LOGIC          := '0';

BEGIN

        ABC: PROCESS(disp_ena, row, column, CLK_50MHz) --Displays menu and game

        BEGIN

                IF(disp_ena = '1' AND Menu_sw = '1') THEN               --Display game
                        IF(row < pixels_y) THEN --purple
                                red <= "01001101";
                                green <= "00000001";
                                blue <= "10001100";
                        ELSIF (row < pixels_y2) THEN --white
                                red <= "11001010";
                                green <= "10110100";
                                blue <= "10110100";
                        ELSIF (row < pixels_y3 and ((column >= 870 and column < 880) or (column >= 980 and column < 990)))
THEN --white bar
                                red <= "11001010";
                                green <= "10110100";
                                blue <= "10110100";
                        ELSIF(row < pixels_y3 - 1 AND column >= green_1 AND column < green_1 + noteWidth) THEN --Begin
Green Notes
                                red <= (OTHERS => '0');
                                green <= (OTHERS => '1');
                                blue <= (OTHERS => '0');
                        ELSIF(row < pixels_y3 - 1 AND column >= green_2 AND column < green_2 + noteWidth) THEN
                                red <= (OTHERS => '0');
```

```
                green <= (OTHERS => '1');
                blue <= (OTHERS => '0');
            ELSIF(row < pixels_y3 - 1 AND column >= green_3 AND column < green_3 + noteWidth) THEN
                red <= (OTHERS => '0');
                green <= (OTHERS => '1');
                blue <= (OTHERS => '0');
            ELSIF(row < pixels_y3 - 1 AND column >= green_4 AND column < green_4 + noteWidth) THEN
                red <= (OTHERS => '0');
                green <= (OTHERS => '1');
                blue <= (OTHERS => '0');
            ELSIF(row < pixels_y3 - 1 AND column >= green_5 AND column < green_5 + noteWidth) THEN
                red <= (OTHERS => '0');
                green <= (OTHERS => '1');
                blue <= (OTHERS => '0');
            ELSIF(row < pixels_y3 - 1 AND column >= green_6 AND column < green_6 + noteWidth) THEN
                red <= (OTHERS => '0');
                green <= (OTHERS => '1');
                blue <= (OTHERS => '0');
            ELSIF(row < pixels_y3 - 1 AND column >= green_7 AND column < green_7 + noteWidth) THEN
                red <= (OTHERS => '0');
                green <= (OTHERS => '1');
                blue <= (OTHERS => '0');
            ELSIF(row < pixels_y3 - 1 AND column >= green_8 AND column < green_8 + noteWidth) THEN --End
Green Notes
                red <= (OTHERS => '0');
                green <= (OTHERS => '1');
                blue <= (OTHERS => '0');
            ELSIF (row < pixels_y3) THEN --dark purple
                red <= "00101111";
                green <= "00101001";
                blue <= "00110011";
            ELSIF (row < pixels_y4) THEN --white
                red <= "11001010";
                green <= "10110100";
                blue <= "10110100";
            ELSIF (row < pixels_y5 and ((column >= 870 and column < 880) or (column >= 980 and column < 990)))
THEN --white bar
                red <= "11001010";
                green <= "10110100";
                blue <= "10110100";
            ELSIF(row < pixels_y5 - 1 AND column >= red_1 AND column < red_1 + noteWidth) THEN --Begin Red
Notes
                red <= (OTHERS => '1');
                green <= (OTHERS => '0');
                blue <= (OTHERS => '0');
            ELSIF(row < pixels_y5 - 1 AND column >= red_2 AND column < red_2 + noteWidth) THEN
                red <= (OTHERS => '1');
                green <= (OTHERS => '0');
                blue <= (OTHERS => '0');
            ELSIF(row < pixels_y5 - 1 AND column >= red_3 AND column < red_3 + noteWidth) THEN
                red <= (OTHERS => '1');
                green <= (OTHERS => '0');
                blue <= (OTHERS => '0');
            ELSIF(row < pixels_y5 - 1 AND column >= red_4 AND column < red_4 + noteWidth) THEN
                red <= (OTHERS => '1');
                green <= (OTHERS => '0');
                blue <= (OTHERS => '0');
            ELSIF(row < pixels_y5 - 1 AND column >= red_5 AND column < red_5 + noteWidth) THEN
                red <= (OTHERS => '1');
                green <= (OTHERS => '0');
                blue <= (OTHERS => '0');
```

```
ELSIF(row < pixels_y5 - 1 AND column >= red_6 AND column < red_6 + noteWidth) THEN
		red <= (OTHERS => '1');
		green <= (OTHERS => '0');
		blue <= (OTHERS => '0');
ELSIF(row < pixels_y5 - 1 AND column >= red_7 AND column < red_7 + noteWidth) THEN
		red <= (OTHERS => '1');
		green <= (OTHERS => '0');
		blue <= (OTHERS => '0');
ELSIF(row < pixels_y5 - 1 AND column >= red_8 AND column < red_8 + noteWidth) THEN --End Red Notes
		red <= (OTHERS => '1');
		green <= (OTHERS => '0');
		blue <= (OTHERS => '0');
ELSIF (row < pixels_y5) THEN --dark purple
		red <= "00101111";
		green <= "00101001";
		blue <= "00110011";
ELSIF (row < pixels_y6) THEN --white
		red <= "11001010";
		green <= "10110100";
		blue <= "10110100";
ELSIF (row < pixels_y7 and ((column >= 870 and column < 880) or (column >= 980 and column < 990)))
THEN --white bar
		red <= "11001010";
		green <= "10110100";
		blue <= "10110100";
ELSIF(row < pixels_y7 - 1 AND column >= yellow_1 AND column < yellow_1 + noteWidth) THEN --Begin
Yellow Notes
		red <= (OTHERS => '1');
		green <= (OTHERS => '1');
		blue <= (OTHERS => '0');
ELSIF(row < pixels_y7 - 1 AND column >= yellow_2 AND column < yellow_2 + noteWidth) THEN
		red <= (OTHERS => '1');
		green <= (OTHERS => '1');
		blue <= (OTHERS => '0');
ELSIF(row < pixels_y7 - 1 AND column >= yellow_3 AND column < yellow_3 + noteWidth) THEN
		red <= (OTHERS => '1');
		green <= (OTHERS => '1');
		blue <= (OTHERS => '0');
ELSIF(row < pixels_y7 - 1 AND column >= yellow_4 AND column < yellow_4 + noteWidth) THEN
		red <= (OTHERS => '1');
		green <= (OTHERS => '1');
		blue <= (OTHERS => '0');
ELSIF(row < pixels_y7 - 1 AND column >= yellow_5 AND column < yellow_5 + noteWidth) THEN
		red <= (OTHERS => '1');
		green <= (OTHERS => '1');
		blue <= (OTHERS => '0');
ELSIF(row < pixels_y7 - 1 AND column >= yellow_6 AND column < yellow_6 + noteWidth) THEN
		red <= (OTHERS => '1');
		green <= (OTHERS => '1');
		blue <= (OTHERS => '0');
ELSIF(row < pixels_y7 - 1 AND column >= yellow_7 AND column < yellow_7 + noteWidth) THEN
		red <= (OTHERS => '1');
		green <= (OTHERS => '1');
		blue <= (OTHERS => '0');
ELSIF(row < pixels_y7 - 1 AND column >= yellow_8 AND column < yellow_8 + noteWidth) THEN --End
Yellow Notes
		red <= (OTHERS => '1');
		green <= (OTHERS => '1');
		blue <= (OTHERS => '0');
ELSIF (row < pixels_y7) THEN --dark purple
		red <= "00101111";
```

```
                        green <= "00101001";
                        blue <= "00110011";
                ELSIF (row < pixels_y8) THEN --white
                        red <= "11001010";
                        green <= "10110100";
                        blue <= "10110100";
                ELSIF (row < pixels_y9 and ((column >= 870 and column < 880) or (column >= 980 and column < 990)))
THEN --white bar
                        red <= "11001010";
                        green <= "10110100";
                        blue <= "10110100";
                ELSIF(row < pixels_y9 - 1 AND column >= blue_1 AND column < blue_1 + noteWidth) THEN --Begin Blue
Notes
                        red <= (OTHERS => '0');
                        green <= (OTHERS => '0');
                        blue <= (OTHERS => '1');
                ELSIF(row < pixels_y9 - 1 AND column >= blue_2 AND column < blue_2 + noteWidth) THEN --End Blue
Notes
                        red <= (OTHERS => '0');
                        green <= (OTHERS => '0');
                        blue <= (OTHERS => '1');
                ELSIF (row < pixels_y9) THEN --dark purple
                        red <= "00101111";
                        green <= "00101001";
                        blue <= "00110011";
                ELSIF (row < pixels_y10) THEN --white
                        red <= "11001010";
                        green <= "10110100";
                        blue <= "10110100";
                ELSIF (row >= pixels_y10) THEN --purple
                        red <= "01001101";
                        green       <= "00000001";
                        blue <= "10001100";
                ELSE
                        red <= (OTHERS => '0');
                        green <= (OTHERS => '0');
                        blue <= (OTHERS => '0');
                END IF;
        ELSE                                    --blanking time
                red <= (OTHERS => '0');
                green <= (OTHERS => '0');
                blue <= (OTHERS => '0');
        END IF;

        IF(disp_ena = '1' AND Menu_sw = '0') THEN               --Display menu
                if (row >= 62 AND row < 124 AND column > 72 AND column < 504) then --G begin
                        red <= (OTHERS => '1');
                        green <= (OTHERS => '1');
                        blue <= (OTHERS => '1');
                elsif (row >= 124 AND row < 186 AND ((column > 72 AND column < 144) OR (column > 432 AND column <
504))) then
                        red <= (OTHERS => '1');
                        green <= (OTHERS => '1');
                        blue <= (OTHERS => '1');
                elsif (row >= 186 AND row < 248 AND ((column > 72 AND column < 144) OR (column > 288 AND column <
360) OR (column > 432 AND column < 504))) then
                        red <= (OTHERS => '1');
                        green <= (OTHERS => '1');
                        blue <= (OTHERS => '1');
                elsif (row >= 248 AND row < 310 AND ((column > 72 AND column < 144) OR (column > 288 AND column <
504))) then --G end
```

```
                                        red <= (OTHERS => '1');
                                        green <= (OTHERS => '1');
                                        blue <= (OTHERS => '1');
                                elsif (row >= 372 AND row < 434 AND ((column > 72 AND column < 504) OR (column > 576 AND column <
1008))) then --U / H begin
                                        red <= (OTHERS => '1');
                                        green <= (OTHERS => '1');
                                        blue <= (OTHERS => '1');
                                elsif (row >= 434 AND row < 558 AND ((column > 432 AND column < 504) OR (column > 720 AND column
< 792))) then
                                        red <= (OTHERS => '1');
                                        green <= (OTHERS => '1');
                                        blue <= (OTHERS => '1');
                                elsif (row >= 558 AND row < 620 AND ((column > 72 AND column < 504) OR (column > 576 AND column <
1008))) then --U / H end
                                        red <= (OTHERS => '1');
                                        green <= (OTHERS => '1');
                                        blue <= (OTHERS => '1');
                                elsif (row >= 682 AND row < 744 AND ((column > 72 AND column < 144) OR (column > 432 AND column <
504) OR (column > 576 AND column < 1008))) then -- I / E begin
                                        red <= (OTHERS => '1');
                                        green <= (OTHERS => '1');
                                        blue <= (OTHERS => '1');
                                elsif (row >= 744 AND row < 806 AND ((column > 72 AND column < 504) OR (column > 576 AND column <
648) OR (column > 720 AND column < 792) OR (column > 936 AND column < 1008))) then
                                        red <= (OTHERS => '1');
                                        green <= (OTHERS => '1');
                                        blue <= (OTHERS => '1');
                                elsif (row >= 806 AND row < 868 AND ((column > 72 AND column < 504) OR (column > 576 AND column <
648) OR (column > 936 AND column < 1008))) then
                                        red <= (OTHERS => '1');
                                        green <= (OTHERS => '1');
                                        blue <= (OTHERS => '1');
                                elsif (row >= 868 AND row < 930 AND ((column > 72 AND column < 144) OR (column > 432 AND column <
504) OR (column > 576 AND column < 648) OR (column > 936 AND column < 1008))) then -- I / E end
                                        red <= (OTHERS => '1');
                                        green <= (OTHERS => '1');
                                        blue <= (OTHERS => '1');
                                elsif (row >= 992 AND row < 1054 AND ((column > 72 AND column < 144) OR (column > 576 AND column
< 1008))) then --T / R begin
                                        red <= (OTHERS => '1');
                                        green <= (OTHERS => '1');
                                        blue <= (OTHERS => '1');
                                elsif (row >= 1054 AND row < 1116 AND ((column > 72 AND column < 504) OR (column > 576 AND
column < 648) OR (column > 720 AND column < 792))) then
                                        red <= (OTHERS => '1');
                                        green <= (OTHERS => '1');
                                        blue <= (OTHERS => '1');
                                elsif (row >= 1116 AND row < 1178 AND ((column > 72 AND column < 504) OR (column > 576 AND
column < 648) OR (column > 720 AND column < 865))) then
                                        red <= (OTHERS => '1');
                                        green <= (OTHERS => '1');
                                        blue <= (OTHERS => '1');
                                elsif (row >= 1178 AND row < 1240 AND ((column > 72 AND column < 144) OR (column > 576 AND
column < 792) OR (column > 864 AND column < 1008))) then -- T / R end
                                        red <= (OTHERS => '1');
                                        green <= (OTHERS => '1');
                                        blue <= (OTHERS => '1');
                                elsif (row >= 1302 AND row < 1364 AND ((column > 72 AND column < 504) OR (column > 576 AND
column < 1008))) then -- A / O begin
                                        red <= (OTHERS => '1');
```

```
                                        green <= (OTHERS => '1');
                                        blue <= (OTHERS => '1');
                                elsif (row >= 1364 AND row < 1488 AND ((column > 72 AND column < 144) OR (column > 288 AND
column < 360) OR (column > 576 AND column < 648) OR (column > 936 AND column < 1008))) then
                                        red <= (OTHERS => '1');
                                        green <= (OTHERS => '1');
                                        blue <= (OTHERS => '1');
                                elsif (row >= 1488 AND row < 1550 AND ((column > 72 AND column < 504) OR (column > 576 AND
column < 1008))) then -- A / O end
                                        red <= (OTHERS => '1');
                                        green <= (OTHERS => '1');
                                        blue <= (OTHERS => '1');
                                elsif (row >= 1612 AND row < 1674 AND column > 72 AND column < 504) then -- R begin
                                        red <= (OTHERS => '1');
                                        green <= (OTHERS => '1');
                                        blue <= (OTHERS => '1');
                                elsif (row >= 1674 AND row < 1736 AND ((column > 72 AND column < 144) OR (column > 216 AND
column < 288))) then
                                        red <= (OTHERS => '1');
                                        green <= (OTHERS => '1');
                                        blue <= (OTHERS => '1');
                                elsif (row >= 1736 AND row < 1798 AND ((column > 72 AND column < 144) OR (column > 216 AND
column < 360))) then
                                        red <= (OTHERS => '1');
                                        green <= (OTHERS => '1');
                                        blue <= (OTHERS => '1');
                                elsif (row >= 1798 AND row < 1860 AND ((column > 72 AND column < 288) OR (column >= 360 AND
column < 504))) then -- R end
                                        red <= (OTHERS => '1');
                                        green <= (OTHERS => '1');
                                        blue <= (OTHERS => '1');
                                else
                                        red <= "01001101";
                                        green <= "00000001";
                                        blue <= "10001100";
                                end if;
                        end if;

                        if CLK_50MHz'event and CLK_50MHz = '1' and Menu_sw = '0' then -- Clock to reset notes when in the menu
                                if counter < "10111110101111000001000000" then
                                        counter <= counter + 1;
                                else
                                        green_1 <= -1510;
                                        green_2 <= -1620;
                                        green_3 <= -1730;
                                        green_4 <= -1950;
                                        green_5 <= -2570;
                                        green_6 <= -3190;
                                        green_7 <= -3990;
                                        green_8 <= -4190;
                                        ---------------------------------
                                        red_1 <= 0;
                                        red_2 <= -110;
                                        red_3 <= -1190;
                                        red_4 <= -1300;
                                        red_5 <= -1840;
                                        red_6 <= -2360;
                                        red_7 <= -2980;
                                        red_8 <= -3690;
                                        ---------------------------------
                                        yellow_1   <= -230;
```

```
                yellow_2   <= -340;
                yellow_3   <= -670;
                yellow_4   <= -970;
                yellow_5   <= -1080;
                yellow_6   <= -2250;
                yellow_7   <= -2870;
                yellow_8   <= -3590;
                --------------------------------
                blue_1 <= -450;
                blue_2 <= -560;
                counter <= (others => '0');
        end if;
end if;

if CLK_50MHz'event and CLK_50MHz = '1' and Menu_sw = '1' then -- Clock to increment note values
        if counter < "1010001011000010 11" then
                counter <= counter + 1;
        else
                green_1 <= green_1 + 1;
                green_2 <= green_2 + 1;
                green_3 <= green_3 + 1;
                green_4 <= green_4 + 1;
                green_5 <= green_5 + 1;
                green_6 <= green_6 + 1;
                green_7 <= green_7 + 1;
                green_8 <= green_8 + 1;
                ------------------------------------------------------------------
                red_1 <= red_1 + 1;
                red_2 <= red_2 + 1;
                red_3 <= red_3 + 1;
                red_4 <= red_4 + 1;
                red_5 <= red_5 + 1;
                red_6 <= red_6 + 1;
                red_7 <= red_7 + 1;
                red_8 <= red_8 + 1;
                ------------------------------------------------------------------
                yellow_1 <= yellow_1 + 1;
                yellow_2 <= yellow_2 + 1;
                yellow_3 <= yellow_3 + 1;
                yellow_4 <= yellow_4 + 1;
                yellow_5 <= yellow_5 + 1;
                yellow_6 <= yellow_6 + 1;
                yellow_7 <= yellow_7 + 1;
                yellow_8 <= yellow_8 + 1;
                ------------------------------------------------------------------
                blue_1 <= blue_1 + 1;
                blue_2 <= blue_2 + 1;
                ------------------------------------------------------------------
                counter <= (others => '0');
        end if;
end if;

END PROCESS;

green_Debounce: process(CLK_50MHz, greenBut) --Green button debouncer
constant init : natural := 3500000;
variable delay : natural range 0 to init := init;
begin
        if rising_edge(CLK_50MHz) then
                if (greenBut = '0') then
                        if (delay = 0) then
```

```
                                        greenBut_debounced <= '1';
                                else
                                        delay := delay - 1;
                                end if;
                        else
                                greenBut_debounced <= '0';
                                delay := init;
                        end if;
        end if;
end process;

red_Debounce: process(CLK_50MHz, redBut) -- Red button debouncer
constant init : natural := 3500000;
variable delay : natural range 0 to init := init;
begin
        if rising_edge(CLK_50MHz) then
                if (redBut = '0') then
                        if (delay = 0) then
                                redBut_debounced <= '1';
                        else
                                delay := delay - 1;
                        end if;
                else
                        redBut_debounced <= '0';
                        delay := init;
                end if;
        end if;
end process;

yellow_Debounce: process(CLK_50MHz, yellowBut) -- Yellow button debouncer
constant init : natural := 3500000;
variable delay : natural range 0 to init := init;
begin
        if rising_edge(CLK_50MHz) then
                if (yellowBut = '0') then
                        if (delay = 0) then
                                yellowBut_debounced <= '1';
                        else
                                delay := delay - 1;
                        end if;
                else
                        yellowBut_debounced <= '0';
                        delay := init;
                end if;
        end if;
end process;

blue_Debounce: process(CLK_50MHz, blueBut) -- Blue button debouncer
constant init : natural := 3500000;
variable delay : natural range 0 to init := init;
begin
        if rising_edge(CLK_50MHz) then
                if (blueBut = '0') then
                        if (delay = 0) then
                                blueBut_debounced <= '1';
                        else
                                delay := delay - 1;
                        end if;
                else
                        blueBut_debounced <= '0';
                        delay := init;
```

```
                        end if;
                end if;
        end process;

BTN_KEY_PRESSED: process(CLK_50MHz, greenBut_debounced) -- Change output "s" based on what buttons were pressed and when, add to score if good button press, subtract from score if bad button press
   variable activated : boolean := false;
   begin
      if rising_edge(CLK_50MHz) then
            if (greenBut_debounced = '1' and Menu_sw = '1' and ((green_1 > 830 and green_1 < 980) or (green_2 > 830 and green_2 < 980) or (green_3 > 830 and green_3 < 980) or (green_4 > 830 and green_4 < 980) or (green_5 > 830 and green_5 < 980) or (green_6 > 830 and green_6 < 980) or (green_7 > 830 and green_7 < 980) or (green_8 > 830 and green_8 < 980))) then
                if not activated then
                        key_pressed <= '1';
                        activated := true;
                        if (add1 /= 26) then
                                add1 <= add1 + 1;
                        end if;
                        s <= std_logic_vector(to_unsigned(add1, s'length));
                else
                   key_pressed <= '0';
                end if;
            elsif (greenBut_debounced = '1' and Menu_sw = '1') then
                 if not activated then
                        key_pressed <= '1';
                        activated := true;
                        if (add1 /= 0) then
                                add1 <= add1 - 1;
                        end if;
                         s <= std_logic_vector(to_unsigned(add1, s'length));
                 else
                                key_pressed <= '0';
                 end if;
            elsif (redBut_debounced = '1' and Menu_sw = '1' and ((red_1 > 830 and red_1 < 980) or (red_2 > 830 and red_2 < 980) or (red_3 > 830 and red_3 < 980) or (red_4 > 830 and red_4 < 980) or (red_5 > 830 and red_5 < 980) or (red_6 > 830 and red_6 < 980) or (red_7 > 830 and red_7 < 980) or (red_8 > 830 and red_8 < 980))) then
                 if not activated then
                        key_pressed <= '1';
                        activated := true;
                        if (add1 /= 26) then
                                add1 <= add1 + 1;
                        end if;
                        s <= std_logic_vector(to_unsigned(add1, s'length));
                else
                   key_pressed <= '0';
                end if;
            elsif (redBut_debounced = '1' and Menu_sw = '1') then
                 if not activated then
                        key_pressed <= '1';
                        activated := true;
                        if (add1 /= 0) then
                                add1 <= add1 - 1;
                        end if;
                        s <= std_logic_vector(to_unsigned(add1, s'length));
                 else
                   key_pressed <= '0';
                 end if;
            elsif (yellowBut_debounced = '1' and Menu_sw = '1' and ((yellow_1 > 830 and yellow_1 < 980) or (yellow_2 > 830 and yellow_2 < 980) or (yellow_3 > 830 and yellow_3 < 980) or (yellow_4 > 830 and yellow_4 < 980) or (yellow_5 > 830 and yellow_5 < 980) or (yellow_6 > 830 and yellow_6 < 980) or (yellow_7 > 830 and yellow_7 < 980) or (yellow_8 > 830 and yellow_8 < 980))) then
                 if not activated then
```

```
                        key_pressed <= '1';
                        activated := true;
                        if (add1 /= 26) then
                                add1 <= add1 + 1;
                        end if;
                        s <= std_logic_vector(to_unsigned(add1, s'length));
                else
                   key_pressed <= '0';
                end if;
        elsif (yellowBut_debounced = '1' and Menu_sw = '1') then
                if not activated then
                        key_pressed <= '1';
                        activated := true;
                        if (add1 /= 0) then
                                add1 <= add1 - 1;
                        end if;
                        s <= std_logic_vector(to_unsigned(add1, s'length));
                else
                   key_pressed <= '0';
                end if;
        elsif (blueBut_debounced = '1' and Menu_sw = '1' and ((blue_1 > 830 and blue_1 < 980) or (blue_2 > 830 and blue_2 < 980))) then
                if not activated then
                        key_pressed <= '1';
                        activated := true;
                        if (add1 /= 26) then
                                add1 <= add1 + 1;
                        end if;
                        s <= std_logic_vector(to_unsigned(add1, s'length));
                else
                   key_pressed <= '0';
                end if;
        elsif (blueBut_debounced = '1' and Menu_sw = '1') then
                if not activated then
                        key_pressed <= '1';
                        activated := true;
                        if (add1 /= 0) then
                                add1 <= add1 - 1;
                        end if;
                        s <= std_logic_vector(to_unsigned(add1, s'length));
                else
                   key_pressed <= '0';
                end if;
        else
                if (Menu_sw = '0') then
                        s <= "00000";
                        add1 <= 0;
                end if;
                    key_pressed <= '0';
                    activated := false;
        end if;
    end if;
  end process;
END behavior;
```

SevenSegmentDisp.vhd

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY SevenSegPort  IS
```

```
        PORT (s :  IN        STD_LOGIC_VECTOR(4 DOWNTO 0)    :=    (OTHERS => '0');
                outA, outB, outC, outD, outE, outF, outG, h, i, j, k, l, m, n    :    OUT STD_LOGIC) ;

END SevenSegPort;

ARCHITECTURE SevenSegmentDisp OF SevenSegPort IS
BEGIN
        --First Seven Segment Display

        outA <= not((s(4) and s(1)) or (not s(3) and s(1)) or (s(4) and not s(2) and s(0)) or (not s(4) and not s(2) and not s(0)) or (not s(4) and
s(2) and s(0)) or (not s(4) and s(3) and not s(1)) or (s(4) and not s(3) and not s(0)));

        outB <= not((not s(4) and not s(2)) or (s(3) and s(2) and not s(1)) or (s(4) and not s(3) and s(0)) or (not s(3) and s(1) and s(0)) or (not
s(4) and s(3) and not s(0)) or (s(4) and not s(3) and s(1)) or (s(2) and not s(1) and not s(0)) or (s(3) and not s(1) and not s(0)));

        outC <= not(s(0) or (s(3) and not s(2)) or (not s(3) and not s(1)) or (s(4) and not s(2)) or (not s(4) and s(2) and s(1)));

        outD <= not((not s(4) and not s(2) and not s(0)) or (not s(3) and s(1) and not s(0)) or (not s(2) and s(1) and not s(0)) or (s(4) and not
s(3) and not s(0)) or (s(3) and s(2) and not s(1)) or (s(3) and s(2) and s(0)) or (s(4) and s(2) and s(1)) or (s(4) and s(3) and s(0)) or (not s(4) and
not s(3) and not s(2) and s(1)) or (not s(4) and s(2) and not s(1) and s(0)));

        outE <= not((not s(4) and not s(2) and not s(0)) or (not s(3) and s(1) and not s(0)) or (not s(2) and s(1) and not s(0)) or (s(4) and not
s(3) and not s(0)) or (not s(4) and s(3) and not s(1) and not s(0)));

        outF <= not((not s(3) and not s(1) and not s(0)) or (s(3) and not s(2) and not s(1)) or (s(3) and not s(2) and not s(0)) or (s(3) and s(2)
and s(1)) or (s(4) and not s(2) and s(1)) or (not s(4) and not s(3) and s(2) and not s(1)) or (not s(4) and not s(3) and s(2) and not s(0)));

        outG <= not((s(3) and not s(1)) or (s(3) and s(2)) or (s(4) and s(1)) or (not s(3) and not s(2) and s(1)) or (not s(3) and s(1) and not s(0))
or (not s(4) and s(2) and not s(1)) or (s(4) and not s(2) and not s(0)));

        --Second Seven Segment Display

        h <= not((not s(4) and not s(3)) OR (not s(3) and s(2)) or (s(4) and s(3)) or (not s(4) and not s(2) and not s(1)));

        i <= '0';

        j <= not((not s(4)) or (not s(3) and not s(2)));

        k <= not((not s(4) and not s(3)) OR (not s(3) and s(2)) or (s(4) and s(3)) or (not s(4) and not s(2) and not s(1)));

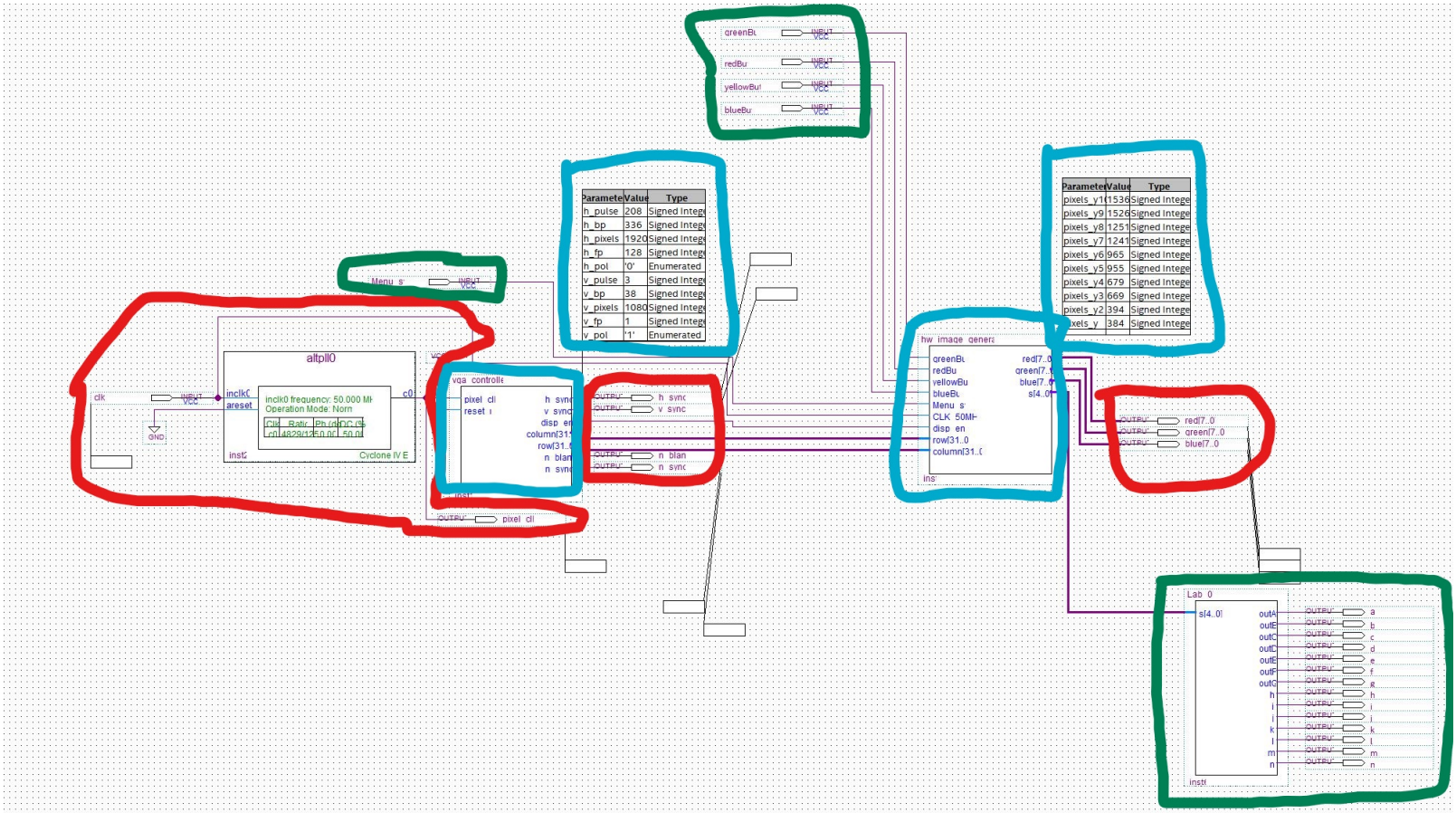
        l <= not((not s(4) and not s(3)) OR (not s(3) and s(2)) or (s(4) and s(3)) or (not s(4) and not s(2) and not s(1)));

   m <= not((not s(4) and not s(3)) or (not s(4) and not s(2) and not s(1)));

        n <= not((s(4) and s(2)) or (s(4) and s(3)));

END SevenSegmentDisp;
```

VGA_with_hw_test_image.bdf



If you cannot see the details of this schematic since it had to be zoomed out, you can always just compare side by side with the schematic in quartus.